EXPERIMENT 11 – CNN FOR IMAGE CLASSIFICATION

1. Aim

To implement a Convolutional Neural Network (CNN) for image classification using a waste classification dataset containing six categories: cardboard, glass, metal, paper, plastic and trash. To train and evaluate the CNN using accuracy, loss, confusion matrix and feature map visualisation.

2. Mathematical Formulae

1. Convolution Operation

$$Y(i,j) = \sum_m \sum_n X(i+m, j+n)K(m,n) + b$$

Where:

- X = Input image
- K = Convolution filter/kernel
- b = Bias
- Y = Output feature map

2. ReLU Activation Function

$$f(x) = \max(0, x)$$

3. Max Pooling

$$P(i,j) = \max_{(m,n)\in R} X(i+m, j+n)$$

4. Softmax Function

$$P(y_i) = \frac{e^{z_i}}{\sum_{j=1}^{C} e^{z_j}}$$

5. Categorical Cross-Entropy Loss

$$L = -\sum_{i=1}^{C} y_i \log(\hat{y}_i)$$

6. Classification Accuracy

$$Accuracy = \frac{Correct\ Predictions}{Total\ Predictions} \times 100$$

3. System Architecture



4. Algorithm

1. Import the required libraries such as TensorFlow, NumPy and Matplotlib.
2. Load the waste classification image dataset containing six categories.
3. Resize all images to \(96 \times 96\) pixels.
4. Normalise the pixel values between 0 and 1.
5. Split the dataset into training, validation and testing sets.
6. Construct a CNN using convolution, pooling, flatten and dense layers.
7. Compile the model using the Adam optimiser and categorical cross-entropy loss.

8. Train the model using the training dataset.
9. Plot training and validation accuracy and loss.
10. Evaluate the model using test data and generate a confusion matrix.
11. Display misclassified images and visualise the convolution layer feature maps.

5. Python Program

```
import os
import numpy as np
import matplotlib.pyplot as plt
import tensorflow as tf

from sklearn.model_selection import train_test_split
from sklearn.metrics import confusion_matrix, ConfusionMatrixDisplay

# 1. Dataset configuration
DATASET_PATH = "waste_dataset"

IMG_SIZE = (96, 96)
BATCH_SIZE = 32
EPOCHS = 10

classes = [
  "cardboard", "glass", "metal",
  "paper", "plastic", "trash"
]

# 2. Load image paths and labels
image_paths = []
labels = []

for label, category in enumerate(classes):

  folder = os.path.join(DATASET_PATH, category)
```

```
    for file in os.listdir(folder):

        path = os.path.join(folder, file)

        if file.lower().endswith(
            (".jpg", ".jpeg", ".png")
        ):
            image_paths.append(path)
            labels.append(label)

image_paths = np.array(image_paths)
labels = np.array(labels)

# 3. Split dataset
X_train, X_temp, y_train, y_temp = train_test_split(
    image_paths,
    labels,
    test_size=0.30,
    random_state=42,
    stratify=labels
)

X_val, X_test, y_val, y_test = train_test_split(
    X_temp,
    y_temp,
    test_size=0.50,
    random_state=42,
    stratify=y_temp
)

# 4. Image preprocessing
def load_image(path, label):

    image = tf.io.read_file(path)
```

```
    image = tf.image.decode_image(
        image,
        channels=3,
        expand_animations=False
    )

    image.set_shape([None, None, 3])

    image = tf.image.resize(image, IMG_SIZE)

    image = image / 255.0

    label = tf.one_hot(label, depth=6)

    return image, label


def create_dataset(paths, labels):

    dataset = tf.data.Dataset.from_tensor_slices(
        (paths, labels)
    )

    dataset = dataset.map(
        load_image,
        num_parallel_calls=tf.data.AUTOTUNE
    )

    return dataset.batch(BATCH_SIZE).prefetch(
        tf.data.AUTOTUNE
    )
```

```
train_ds = create_dataset(X_train, y_train)
val_ds = create_dataset(X_val, y_val)
test_ds = create_dataset(X_test, y_test)

# 5. Build CNN model
model = tf.keras.Sequential([

  tf.keras.layers.Input(shape=(96, 96, 3)),

  tf.keras.layers.Conv2D(
    32, (3, 3), activation="relu",
    padding="same", name="conv1"
  ),
  tf.keras.layers.MaxPooling2D((2, 2)),

  tf.keras.layers.Conv2D(
    64, (3, 3), activation="relu",
    padding="same"
  ),
  tf.keras.layers.MaxPooling2D((2, 2)),

  tf.keras.layers.Conv2D(
    128, (3, 3), activation="relu",
    padding="same"
  ),
  tf.keras.layers.MaxPooling2D((2, 2)),

  tf.keras.layers.Flatten(),

  tf.keras.layers.Dense(128, activation="relu"),

  tf.keras.layers.Dropout(0.3),

  tf.keras.layers.Dense(6, activation="softmax")
```

```
])

# 6. Compile model
model.compile(
  optimizer="adam",
  loss="categorical_crossentropy",
  metrics=["accuracy"]
)

model.summary()

# 7. Train model
history = model.fit(
  train_ds,
  validation_data=val_ds,
  epochs=EPOCHS
)

# 8. Plot accuracy
plt.figure(figsize=(8, 5))

plt.plot(history.history["accuracy"], label="Training Accuracy")
plt.plot(history.history["val_accuracy"], label="Validation Accuracy")

plt.title("CNN Accuracy")
plt.xlabel("Epochs")
plt.ylabel("Accuracy")
plt.legend()
plt.show()

# 9. Plot loss
plt.figure(figsize=(8, 5))

plt.plot(history.history["loss"], label="Training Loss")
```

```
plt.plot(history.history["val_loss"], label="Validation Loss")

plt.title("CNN Loss")
plt.xlabel("Epochs")
plt.ylabel("Loss")
plt.legend()
plt.show()

# 10. Evaluate model
test_loss, test_accuracy = model.evaluate(test_ds)

print("Test Loss:", test_loss)
print("Test Accuracy:", test_accuracy)

# 11. Confusion matrix
y_true = y_test

predictions = model.predict(test_ds)

y_pred = np.argmax(predictions, axis=1)

cm = confusion_matrix(y_true, y_pred)

display = ConfusionMatrixDisplay(
   confusion_matrix=cm,
   display_labels=classes
)

display.plot(cmap="Blues", xticks_rotation=45)

plt.title("Confusion Matrix")
plt.show()

# 12. Display misclassified images
```

```
misclassified = np.where(y_true != y_pred)[0]

plt.figure(figsize=(12, 6))

for i, index in enumerate(misclassified[:6]):

  image = tf.keras.utils.load_img(
    X_test[index],
    target_size=IMG_SIZE
  )

  plt.subplot(2, 3, i + 1)
  plt.imshow(image)

  plt.title(
    "Actual: " + classes[y_true[index]] +
    "\nPredicted: " + classes[y_pred[index]]
  )

  plt.axis("off")

plt.tight_layout()
plt.show()

# 13. Visualise feature maps
feature_model = tf.keras.Model(
  inputs=model.inputs,
  outputs=model.get_layer("conv1").output
)

sample_image = tf.keras.utils.load_img(
  X_test[0],
  target_size=IMG_SIZE
)
```

```
sample_array = tf.keras.utils.img_to_array(
    sample_image
) / 255.0

sample_array = np.expand_dims(sample_array, axis=0)

feature_maps = feature_model.predict(sample_array)

plt.figure(figsize=(10, 10))

for i in range(16):

    plt.subplot(4, 4, i + 1)

    plt.imshow(feature_maps[0, :, :, i], cmap="viridis")

    plt.axis("off")

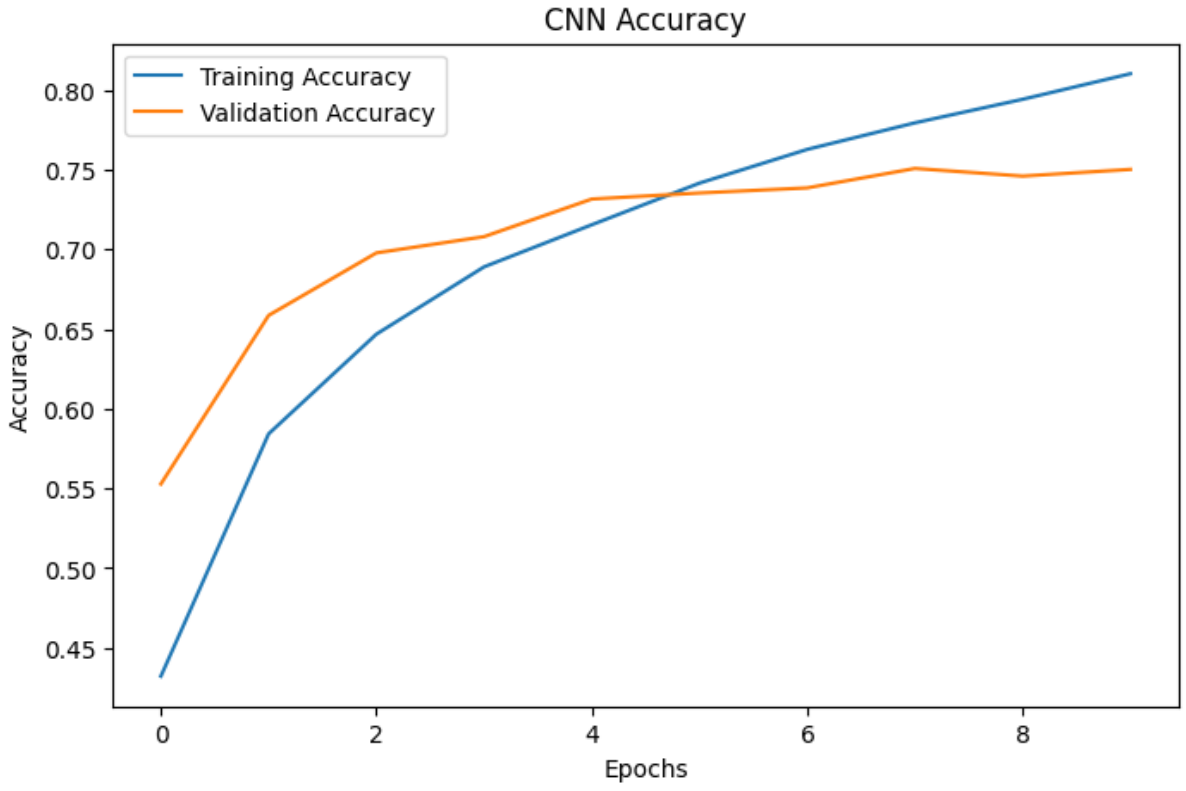
plt.suptitle("CNN Feature Maps")
plt.show()
```

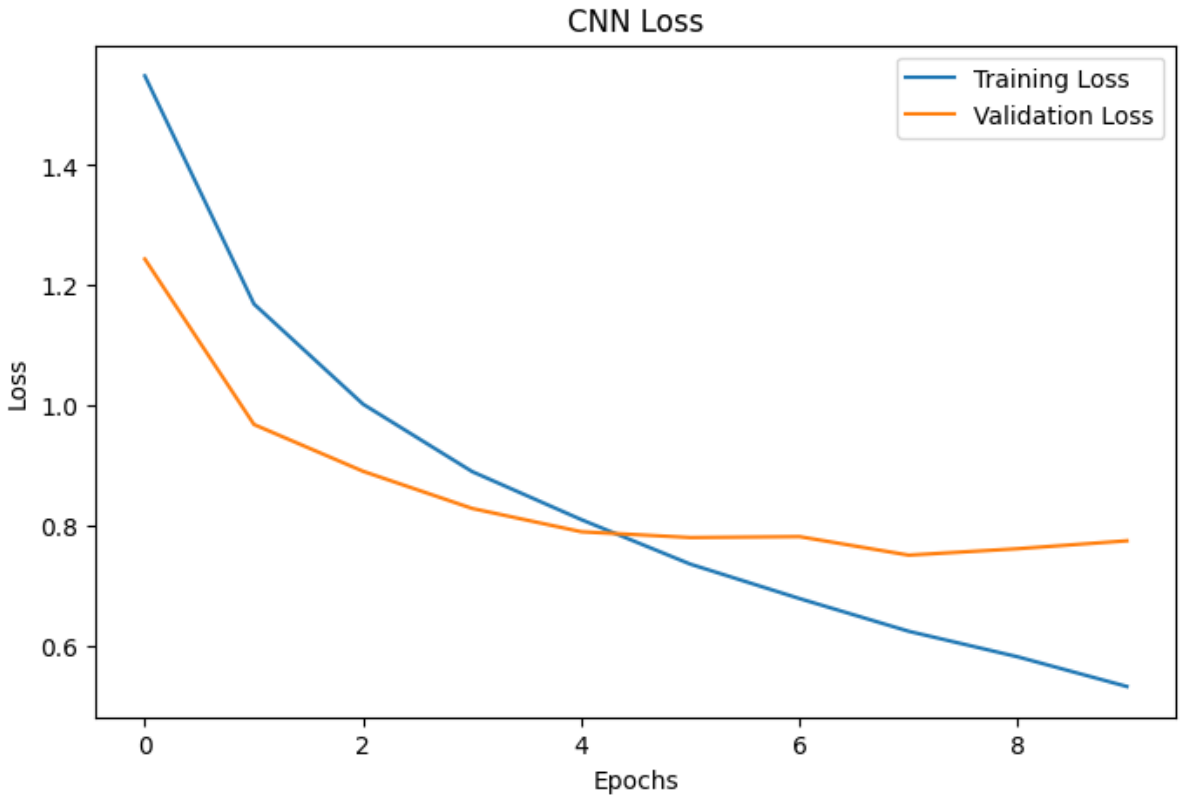
6. Output

```
Downloading data from https://www.cs.toronto.edu/~kriz/cifar-10-python.tar.gz
170498071/170498071 ━━━━━━━━━━━━━━━━━━━━ 1824s 11us/step
Model: "sequential"
```

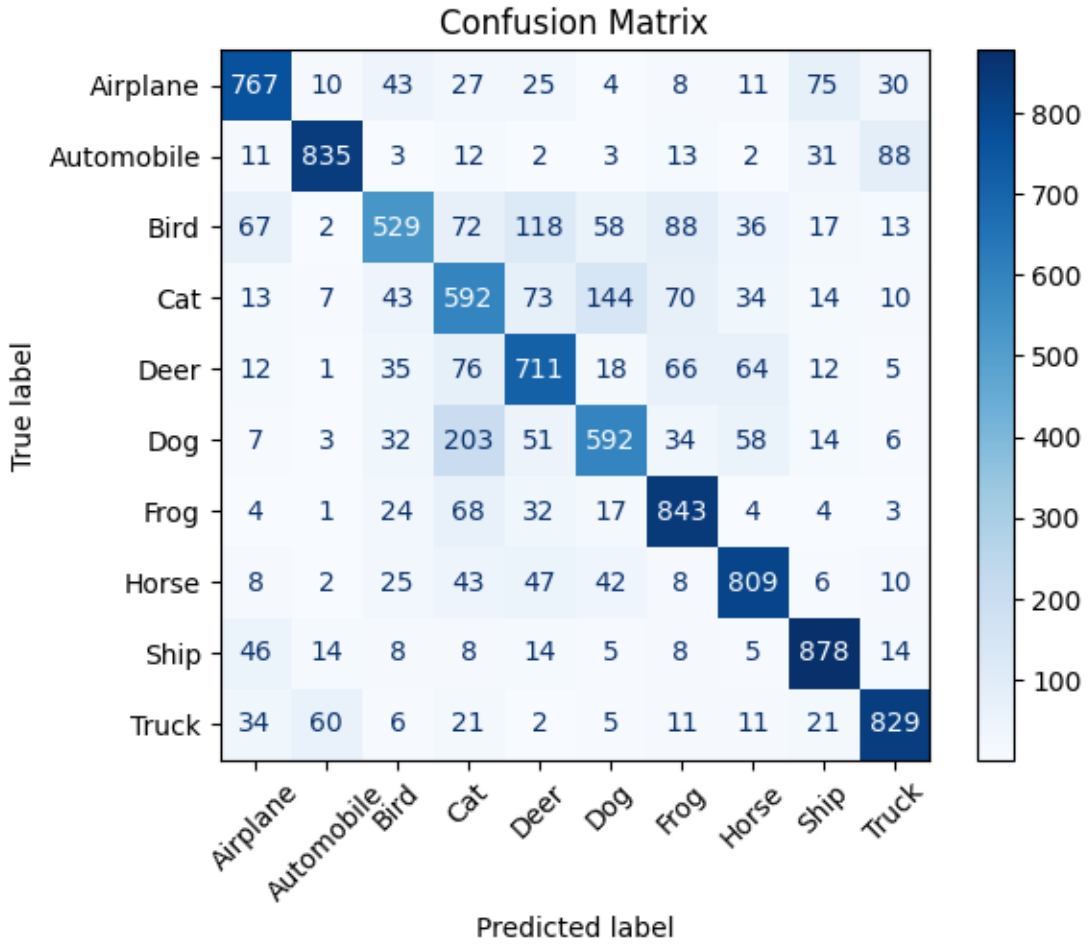
Layer (type)	Output Shape	Param #
conv1 (Conv2D)	(None, 32, 32, 32)	896
max_pooling2d (MaxPooling2D)	(None, 16, 16, 32)	0
conv2d (Conv2D)	(None, 16, 16, 64)	18,496
max_pooling2d_1 (MaxPooling2D)	(None, 8, 8, 64)	0
conv2d_1 (Conv2D)	(None, 8, 8, 128)	73,856
max_pooling2d_2 (MaxPooling2D)	(None, 4, 4, 128)	0
flatten (Flatten)	(None, 2048)	0
dense (Dense)	(None, 128)	262,272
dropout (Dropout)	(None, 128)	0
dense_1 (Dense)	(None, 10)	1,290

```
Total params: 356,810 (1.36 MB)
Trainable params: 356,810 (1.36 MB)
Non-trainable params: 0 (0.00 B)
Epoch 1/10
1407/1407 ━━━━━━━━━━━━━━━━━━━━ 103s 72ms/step - accuracy: 0.4322 - loss: 1.5489 - val_accuracy: 0.5528 - val_loss: 1.2438
Epoch 2/10
1407/1407 ━━━━━━━━━━━━━━━━━━━━ 142s 72ms/step - accuracy: 0.5843 - loss: 1.1689 - val_accuracy: 0.6586 - val_loss: 0.9682
Epoch 3/10
1407/1407 ━━━━━━━━━━━━━━━━━━━━ 146s 75ms/step - accuracy: 0.6468 - loss: 1.0021 - val_accuracy: 0.6978 - val_loss: 0.8902
Epoch 4/10
1407/1407 ━━━━━━━━━━━━━━━━━━━━ 103s 73ms/step - accuracy: 0.6890 - loss: 0.8897 - val_accuracy: 0.7080 - val_loss: 0.8283
Epoch 5/10
1407/1407 ━━━━━━━━━━━━━━━━━━━━ 101s 72ms/step - accuracy: 0.7155 - loss: 0.8097 - val_accuracy: 0.7316 - val_loss: 0.7895
Epoch 6/10
1407/1407 ━━━━━━━━━━━━━━━━━━━━ 145s 74ms/step - accuracy: 0.7416 - loss: 0.7355 - val_accuracy: 0.7354 - val_loss: 0.7799
Epoch 7/10
1407/1407 ━━━━━━━━━━━━━━━━━━━━ 103s 73ms/step - accuracy: 0.7628 - loss: 0.6782 - val_accuracy: 0.7386 - val_loss: 0.7814
Epoch 8/10
1407/1407 ━━━━━━━━━━━━━━━━━━━━ 101s 72ms/step - accuracy: 0.7794 - loss: 0.6237 - val_accuracy: 0.7508 - val_loss: 0.7506
Epoch 9/10
1407/1407 ━━━━━━━━━━━━━━━━━━━━ 102s 73ms/step - accuracy: 0.7942 - loss: 0.5812 - val_accuracy: 0.7460 - val_loss: 0.7613
Epoch 10/10
1407/1407 ━━━━━━━━━━━━━━━━━━━━ 101s 72ms/step - accuracy: 0.8102 - loss: 0.5317 - val_accuracy: 0.7502 - val_loss: 0.7744
```






```
313/313 ━━━━━━━━━━━━━━━━━━━━ 10s 32ms/step - accuracy: 0.7385 - loss: 0.8081
Test Loss: 0.8080877065658569
Test Accuracy: 0.7384999990463257
313/313 ━━━━━━━━━━━━━━━━━━━━ 6s 19ms/step
```


Confusion Matrix
Airplane 767 10 43 27 25 4 8 11 75 30
Automobile 11 835 3 12 2 3 13 2 31 88
Bird 67 2 529 72 118 58 88 36 17 13
Cat 13 7 43 592 73 144 70 34 14 10
Deer 12 1 35 76 711 18 66 64 12 5
Dog 7 3 32 203 51 592 34 58 14 6
Frog 4 1 24 68 32 17 843 4 4 3
Horse 8 2 25 43 47 42 8 809 6 10
Ship 46 14 8 8 14 5 8 5 878 14
Truck 34 60 6 21 2 5 11 11 21 829
800
700
600
500
400
300
200
100
True label
Airplane
Automobile
Bird
Cat
Deer
Dog
Frog
Horse
Ship
Truck
Predicted label


Actual: Cat
Predicted: Dog

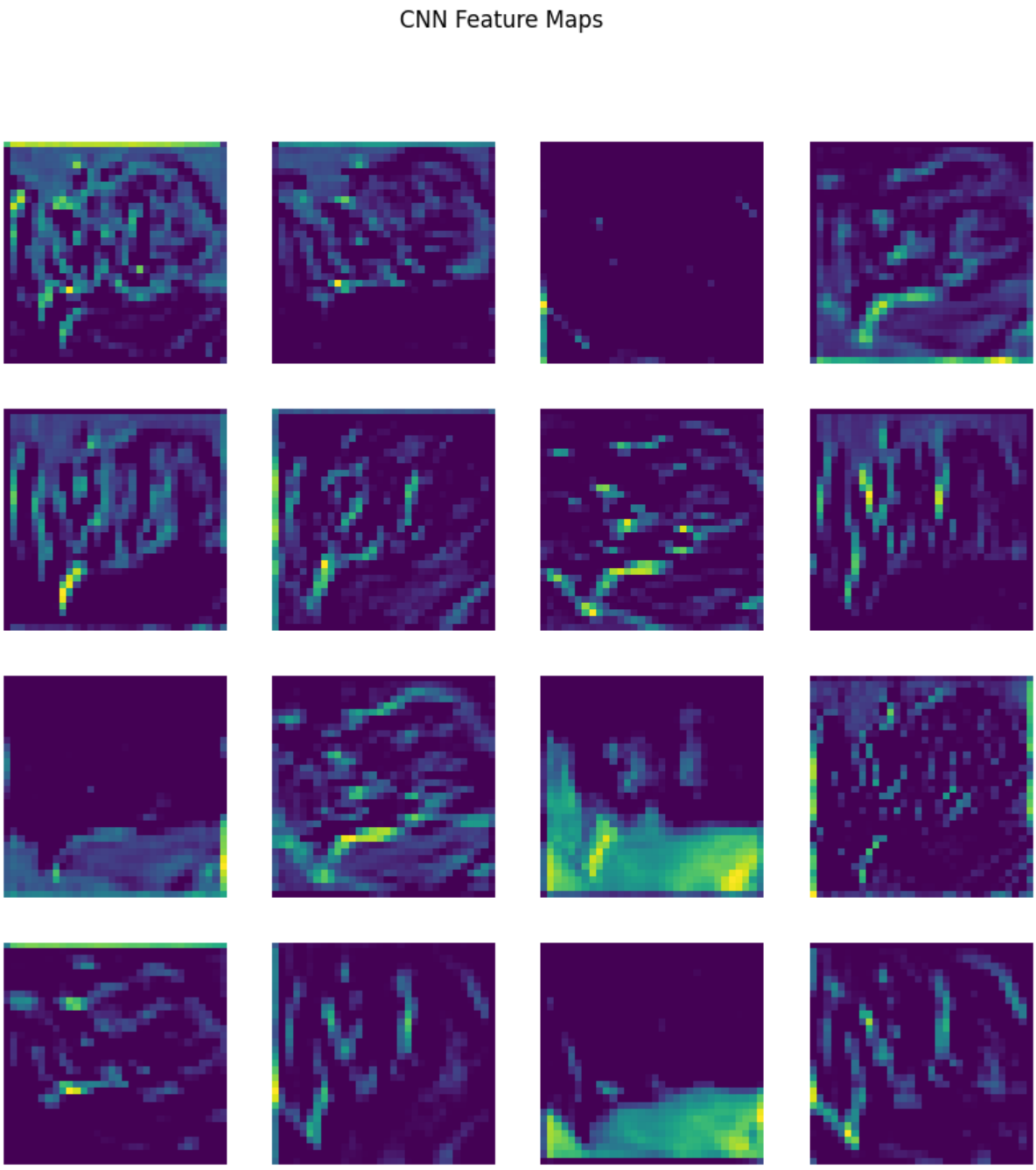

Actual: Airplane
Predicted: Ship


Actual: Automobile
Predicted: Truck


Actual: Dog
Predicted: Deer


Actual: Dog
Predicted: Cat


Actual: Bird
Predicted: Ship



7. Result

Thus, a Convolutional Neural Network was implemented successfully for waste image classification. The model was trained using six waste categories and evaluated using accuracy, loss, confusion matrix and feature map visualisation.